



Pokédex

Eva Gallardo Romero

14 Octubre 2025 a 23 Octubre 2025

Índice

Enlace al repositorio de GitHub	2
1. Idea general	2
2. Investigación y referencias	2
3. Diagrama de clases	2
4. Diseño	3
4.1. Icono de la app	3
4.2. Pantalla de bienvenida (landing page)	4
4.3. Vista general de la Pokédex	5
4.4. Pantalla de tipos de Pokémon	5
5. Requisitos del sistema	7
6. Documentación técnica	8
6.1. API.java	8
6.2. Pokemon.java	12
6.3. TipoPokemonEnum.java	14
6.4. LandingActivity.java	16
6.5. GeneralPokedex.java	17
6.6. InfoPokemon.java	19
6.7. PokemonAdapter.java	21
6.8. AndroidManifest.xml	23
6.9. Layouts XML	24
6.10. colors.xml	29

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo y descargable en el siguiente enlace: [Pokedex](#)

1. Idea general

Mi idea inicial para la Pokédex es crear una aplicación con una pantalla de bienvenida (landing page) donde aparezca un botón que diga “*Accede a tu Pokédex*”.

Cuando entres, se mostrará una vista general con todos los Pokemones junto con su **ID**, **nombre** y una **imagen**.

Si haces clic en alguno, aparecerá una vista más detallada con más info: su **historia**, **ID**, **nombre** y **tipo**...

Todo esto lo sacaré mediante una llamada a una **API**(PokeAPI).

2. Investigación y referencias

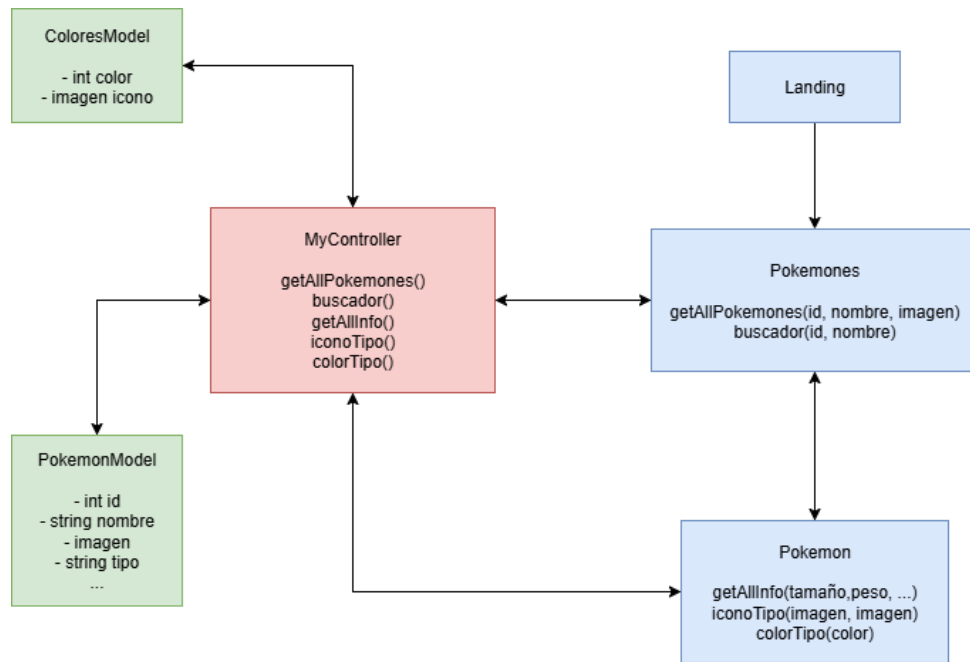
Antes de ponerme a **diseñar**, estuve mirando varias cosas para **coger ideas** y decidir cómo quería que se viera mi **Pokédex**.

Busque en **internet** la información **basica** que suelen contener las **pokedex**, **funcionalidades** que tienen...

Una vez tuve claro lo que tenia que contener mi **aplicación**, me puse con el tema de **diseño** miré cómo se organizaban las **tarjetas de Pokémon** y como se mostraba la **informacion** sobre ellos en otros proyectos para **inspirarme**.

3. Diagrama de clases

En este punto empecé a hacerme un **diagrama de clases** con las **relaciones** que habra entre las principales partes de la **aplicación**, no representa la **implementación final** ya que incluye únicamente las **clases** más estrictamente relacionadas con la **app** y la **generación procedimental de contenido**, obviando **getters** y **setters**.



4. Diseño

Aquí muestro cómo fui dando forma a la **app** con diferentes **pantallas** y **bocetos**. Dándole poco a poco el **enfoque** que quería a mi **pokedex**, decantandome por **tonos** mas **pastel** y una **estetica retro**

Decidi hacerme mi propia **guia de colores** para los diferentes **tipos de pokemones**, cogiendo de referencia el **color original** y **aclarandolo**, para que siguiese la estetica de tonos pasteles que tiene la **aplicación**

Normal #adab8d 	Fuego #ff7111 	Agua #717fff 	Eléctrico #ffd525 	Planta #3bcb01 	Hielo #88dbec
Lucha #c70000 	Veneno #ae01cb 	Tierra #e5bc61 	Volador #a890fe 	Psíquico #fe509b 	Bicho #9cc31f
Roca #c49100 	Fantasma #663993 	Dragon #6622ee 	Siniestro #484848 	Acero #a6a8c6 	Hada #f9a5d1

Colores originales

Normal #d6d5c6 	Fuego #ffc097 	Agua #b8bfff 	Eléctrico #ffea92 	Planta #9de57f 	Hielo #c3edf5
Lucha #e37f7f 	Veneno #d67fe5 	Tierra #f2ddb0 	Volador #d3c7fe 	Psíquico #fea7cd 	Bicho #cde18f
Roca #e1c87f 	Fantasma #b29cc9 	Dragon #b290f6 	Siniestro #a3a3a3 	Acero #d2d3e2 	Hada #fcd2e8

Colores aplicación

4.1. Icono de la app

Quise implementar tambien un **icono personalizado** para mi **aplicacion**, descartando **diseños** demasiado **cargados** o muy **futuristas** para la estetica que yo buscaba. Decidiendome asi por uno **simple** y con un diseño **clasico y limpio**



Elegido:



Descartados:



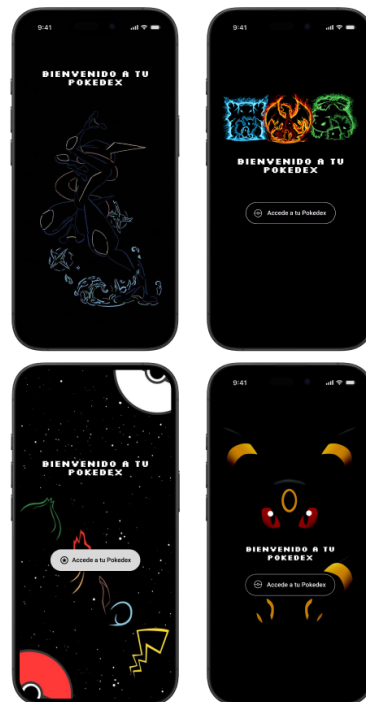
No tendra ninguna **funcionalidad especial**, fuera de la **estetica externa** de la **app** y de **iniciar la aplicacion** mostrandote la pantalla de landing

4.2. Pantalla de bienvenida (landing page)

A partir de esta **pantalla** me guie para **diseñar** toda la **aplicación**, logrando asi conseguir una **armonia**. Para ello tuve que **descartar** otros diseños, mas **minimalistas** y **oscuros** ya que queria que la aplicación fuese algo mas **amigable** con un toque **retro** y **tonos pastel**



Diseño elegido.

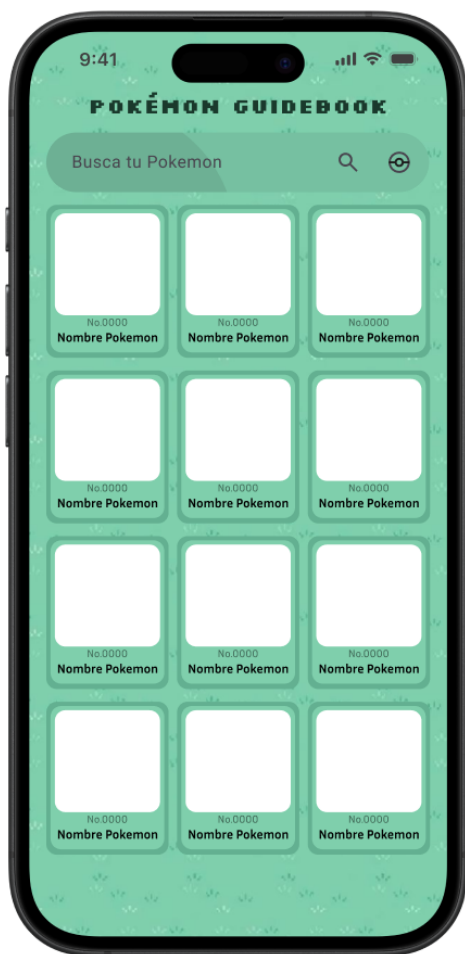


Diseños descartados.

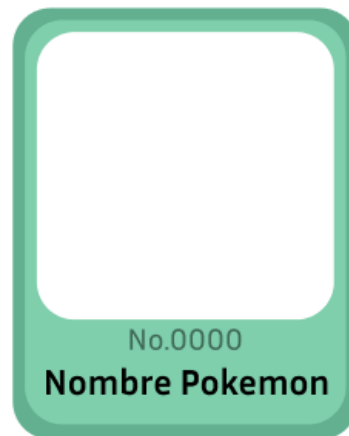
En esta pantalla se le da la **bienvenida** al **usuario**, el cual no necesita **registro** ya que simplemente es una aplicacion en la que podra **buscar informacion**, pudiendo asi empezar a **navegar** en ella clicando en el boton de *Accede a tu Pokédex*, donde sera redirigido a la siguiente pantalla donde vera una **vista general** de la **Pokédex**

4.3. Vista general de la Pokédex

En esta **vista general** de la **pokedex**, utilice el mismo **fondo** que en la pantalla de **landing**, para que todo siguiese la misma **estetica**, y por eso mismo decidi utilizar **tonos verde pastel** para las **tarjetas** donde mostrare los **pokemones** y el **buscador**.



Vista general



Tarjeta ampliada

Esta pantalla incluye un **buscador**, donde el usuario podra buscar un **pokemon** ya sea por **nombre**, o por numero. Tambien al clicar sobre una **tarjeta de pokemon**, se le llevara a otra pantalla con **informacion mas ampliada** sobre ese pokemon

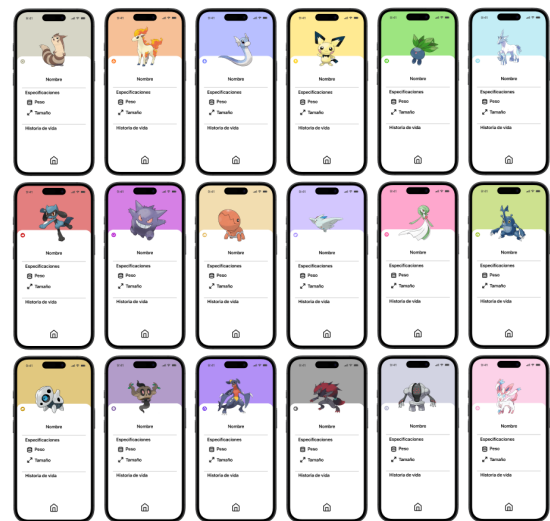
4.4. Pantalla de tipos de Pokémon

En esta pantalla se muestra un **Pokemon** de manera mas **detallada**, mostrando una **imagen** de él, su **tipo**, **peso**, **tamaño**, **hitoria de vida**... Y la parte superior cambia

su **color** dependiendo de su **tipo**.



Vista de un Pokémon individual



Diseños con distintos tipos de Pokémon

Además, tiene un **botón** que te permite volver a la **vista general** para que puedas seguir **explorando** fácilmente otros **Pokémon**.

5. Requisitos del sistema

ID	Requisito funcional
RF01	Mostrar una lista de Pokémon en tarjetas reutilizables mediante un <code>RecyclerView</code> .
RF02	Permitir la búsqueda de Pokémon por nombre utilizando una barra de búsqueda (<code>SearchView</code>).
RF03	Mostrar pantalla de detalle con imagen, nombre, peso, tamaño, historia y tipos al seleccionar un Pokémon.
RF04	Cargar imágenes desde URL utilizando la librería Glide.
RF05	Aplicar color de fondo e íconos según el tipo principal del Pokémon usando <code>TipoPokemonEnum</code> .
RF06	Permitir volver desde la pantalla de detalle a la lista principal mediante un botón de regreso.
RF07	Mostrar una pantalla de bienvenida con botón de acceso y barra de progreso al iniciar la aplicación.
RF08	Mantener consistencia visual en colores, íconos e imágenes en todas las vistas.

ID	Requisito no funcional
RNF01	Las imágenes deben cargarse eficientemente y sin animaciones innecesarias para mejorar el rendimiento.
RNF02	El diseño debe ser claro y adaptable, con márgenes, colores y tipografía legibles.
RNF03	El código debe estar organizado en clases separadas con responsabilidades claras.
RNF04	Los recursos visuales deben definirse en archivos XML para facilitar personalización y mantenimiento.
RNF05	La aplicación debe usar componentes modernos de Android y soportar temas oscuros y claros.
RNF06	La navegación entre pantallas debe ser clara, directa e intuitiva.
RNF07	Todos los elementos visuales deben tener <code>contentDescription</code> para mejorar la accesibilidad.
RNF08	Los textos deben estar definidos en <code>strings.xml</code> para permitir traducción a otros idiomas.

6. Documentación técnica

Este anexo agrupa la documentación por archivos, separandola en la **ubicacion** donde se encuentra, una **descripcion general** y las **funciones** explicadas de manera individual y su **codigo** adjuntado y comentado. Ademas incluye **XML** con los valores que se definen en cada objeto utilizado.

6.1. API.java

6.1.1. Ubicación

```
com.example.pokedex.Controller
```

6.1.2. Descripción general

Controlador de red que se comunica con la **PokeAPI**. Obtiene la lista de **URLs** de todos los **Pokémon**, descarga **datos individuales** (nombre, imagen, id, peso, altura, tipos) y consulta el **endpoint** de especie para extraer la **historia** en español. Almacena todos los objetos **Pokemon** en **myPokedex** y sincroniza **múltiples peticiones** para invocar un callback cuando todo haya terminado.

6.1.3. Funciones

obtainAllPokemon(Context ct, Runnable onComplete) Obtiene la lista de URLs de todos los Pokémon desde la PokeAPI y luego invoca **gatherAllPokemonInfo** para descargar los datos individuales.

```
1 // Inicia la descarga del listado de Pokmon y prepara las URLs para la descarga
  detallada.
2 public static void obtainAllPokemon(Context ct, Runnable onComplete) {
3     RequestQueue queue = Volley.newRequestQueue(ct); // Crea la cola de peticiones con
      el contexto
4     String url = "https://pokeapi.co/api/v2/pokemon?limit=1028&offset=0"; // Endpoint
      para obtener todas las entradas
5
6     // Peticin GET para obtener el JSON con la lista de Pokmon
7     StringRequest stringRequest = new StringRequest(Request.Method.GET, url,
8         response -> {
9         try {
10             JSONObject jsonObj = new JSONObject(response); // Parseo del JSON de
              respuesta
11             // Recorre 'results' y almacena cada 'url' en el array fullURLList
12             for (int i = 0; i < jsonObj.getJSONArray("results").length(); i++) {
13                 fullURLList[i] = jsonObj.getJSONArray("results").getJSONObject(i).
                  getString("url");
14             }
15             // Llama a la funcin que descargar cada Pokmon individualmente
```

```

16         gatherAllPokemonInfo(ct, onComplete);
17     } catch (JSONException e) {
18         e.printStackTrace(); // Registro de error de parseo
19     }
20 },
21 error -> {
22     // error de la petición inicial
23 }
24 );
25 queue.add(stringRequest); // Envía la petición a la cola
26 }

```

gatherAllPokemonInfo(Context ct, Runnable onComplete) Recorre cada URL obtenida, **descarga los datos** del Pokémon (nombre, imagen, id, peso, altura, tipos) y su **historia** en español. Crea **objetos** Pokemon y los **añade** a myPokedex. Usa **AtomicInteger** para detectar cuándo todas las **peticiones** han **finalizado** y ejecutar el **callback**.

```

1 // Descarga detallada: por cada URL solicita datos del Pokmon y luego su recurso de
  // especies.
2 public static void gatherAllPokemonInfo(Context ct, Runnable onComplete) {
3     myPokedex.clear(); // Limpia la lista antes de poblarla de nuevo
4     RequestQueue queue = Volley.newRequestQueue(ct); // Nueva cola de peticiones
5     queue.stop(); // Se detiene para añadir peticiones antes de arrancar
6
7     // Calcula cuantas URLs válidas hay (filtra nulos)
8     final int total = (int) java.util.Arrays.stream(fullURLList)
9         .filter(java.util.Objects::nonNull)
10        .count();
11
12     AtomicInteger completed = new AtomicInteger(0); // Contador seguro en concurrencia
13
14     // Itera sobre todas las URLs y añade peticiones para cada Pokmon
15     for (int i = 0; i < fullURLList.length; i++) {
16         String url = fullURLList[i];
17         if (url == null) continue; // Salta entradas nulas
18
19         // Petición para obtener el JSON del Pokmon individual
20         queue.add(new StringRequest(Request.Method.GET, url,
21             response -> {
22                 try {
23                     JSONObject jsonObj = new JSONObject(response); // Parseo del JSON del
24                                     // Pokmon
25
26                     // Extrae campos principales
27                     String name = jsonObj.getString("name");
28                     int id = jsonObj.getInt("id");
29                     String image = jsonObj.getJSONObject("sprites")

```

```

30         .getJSONObject("official-artwork")
31         .getString("front_default");
32     double weight = jsonObj.getDouble("weight") / 10.0; // Convertir a kg
33     double height = jsonObj.getDouble("height") / 10.0; // Convertir a m
34
35     // Construye la lista de tipos
36     ArrayList<String> tipos = new ArrayList<>();
37     for (int j = 0; j < jsonObj.getJSONArray("types").length(); j++) {
38         JSONObject tipoObj = jsonObj.getJSONArray("types").getJSONObject(j).
39             getJSONObject("type");
40         tipos.add(tipoObj.getString("name"));
41     }
42
43     // Construye URL de species para obtener flavor text en español
44     String speciesUrl = "https://pokeapi.co/api/v2/pokemon-species/" +
45         id;
46     queue.add(new StringRequest(Request.Method.GET, speciesUrl,
47         speciesResponse -> {
48         try {
49             JSONObject speciesObj = new JSONObject(speciesResponse);
50             String historia = "Sin historia"; // Valor por defecto
51
52             // Busca la primera entrada en español y limpia caracteres
53             // especiales
54             for (int k = 0; k < speciesObj.getJSONArray("
55                 flavor_text_entries").length(); k++) {
56                 JSONObject entry = speciesObj.getJSONArray("
57                     flavor_text_entries").getJSONObject(k);
58                 if (entry.getJSONObject("language").getString("name").
59                     equals("es")) {
60                     historia = entry.getString("flavor_text").replace(
61                         "\n", " ").replace("\f", " ");
62                     break; // Sale al encontrar la primera en español
63                 }
64             }
65
66             // Crea el objeto Pokemon y lo aade a la lista global
67             Pokemon pokemon = new Pokemon(name, image, id, weight,
68                 height, historia);
69             pokemon.setTipos(tipos);
70             myPokedex.add(pokemon);
71
72         } catch (JSONException e) {
73             e.printStackTrace(); // Error al parsear speciesResponse
74         } finally {
75             // Incrementa el contador; si llega a 'total', ejecuta el
76             // callback
77             if (completed.incrementAndGet() == total && onComplete !=

```

```

69         null) {
70             onComplete.run();
71         }
72     },
73     error -> {
74         // En caso de error al obtener species, incrementa contador
75         // para no bloquear
76         if (completed.incrementAndGet() == total && onComplete !=
77             null) {
78             onComplete.run();
79         }
80     }));
81
82     } catch (JSONException e) {
83         e.printStackTrace(); // Error al parsear el JSON del Pokemon
84         // Asegura que errores cuentan para el final
85         if (completed.incrementAndGet() == total && onComplete != null) {
86             onComplete.run();
87         }
88     },
89     error -> {
90         // Error en la peticin del Pokemon; cuenta igualmente para no bloquear la
91         // finalizacin
92         if (completed.incrementAndGet() == total && onComplete != null) {
93             onComplete.run();
94         }
95     }));
96
97     }
98
99     queue.start(); // Inicia la ejecucin de todas las peticiones aadidas
100 }

```

getMyPokedex() Devuelve la lista interna myPokedex con todos los objetos Pokemon descargados.

```

1 // Getter pblico que devuelve la lista esttica myPokedex.
2 // NOTA: devuelve la referencia directa; cdigo externo puede modificarla.
3 public static ArrayList<Pokemon> getMyPokedex() {
4     return myPokedex;
5 }

```

6.2. Pokemon.java

6.2.1. Ubicación

`com.example.pokedex.Model`

6.2.2. Descripción general

Clase modelo que representa a un **Pokémon** con **atributos básicos** y **getters/setters**. Usada por el controlador para **crear instancias** y por adaptadores/vistas para leer datos.

6.2.3. Funciones

Constructor Inicializa los **atributos principales** del Pokémon: nombre, imagen, número, peso, tamaño, historia y lista de tipos vacía.

```
1 public Pokemon(String nombre, String imagen, int numero, double peso, double tamao,
2     String historia) {
3     this.name = nombre;
4     this.image = imagen;
5     this.number = numero;
6     this.peso = peso;
7     this.tamao = tamao;
8     this.historia = historia;
9     this.tipos = new ArrayList<>();
10 }
```

getName / setName Accede o modifica el nombre del Pokémon.

```
1 public String getName() {
2     return name;
3 }
4
5 public void setName(String name) {
6     this.name = name;
7 }
```

getImage / setImage Accede o modifica la URL de la imagen oficial.

```
1 public String getImage() {
2     return image;
3 }
4
5 public void setImage(String image) {
6     this.image = image;
7 }
```

getNumber / setNumber Accede o modifica el número identificador del Pokémon.

```
1 public int getNumber() {  
2     return number;  
3 }  
4  
5 public void setNumber(int number) {  
6     this.number = number;  
7 }
```

getPeso / setPeso Accede o modifica el peso del Pokémon en kilogramos.

```
1 public double getPeso() {  
2     return peso;  
3 }  
4  
5 public void setPeso(double peso) {  
6     this.peso = peso;  
7 }
```

getTamaño / setTamaño Accede o modifica la altura del Pokémon en metros.

```
1 public double getTamao() {  
2     return tamao;  
3 }  
4  
5 public void setTamao(double tamao) {  
6     this.tamao = tamao;  
7 }
```

getHistoria / setHistoria Accede o modifica la historia en español del Pokémon.

```
1 public String getHistoria() {  
2     return historia;  
3 }  
4  
5 public void setHistoria(String historia) {  
6     this.historia = historia;  
7 }
```

getTipos / setTipos Accede o modifica la lista de tipos del Pokémon (agua, fuego, etc.).

```
1 public ArrayList<String> getTipos() {  
2     return tipos;  
3 }  
4
```

```

5 public void setTipos(ArrayList<String> tipos) {
6     this.tipos = tipos;
7 }

```

6.3. TipoPokemonEnum.java

Ubicación

com.example.pokedex.Model

6.3.1. Descripción general

Enum que mapea cada **tipo** de **Pokémon** (nombre usado por la API) a un **recurso** de imagen (mipmap) y a un **color** (colors.xml). **Facilita** la aplicación de **estilo visual** en InfoPokemon.

6.3.2. Funciones

Declaración de enums Cada **tipo** de Pokémon se define con su nombre (en inglés), recurso de **imagen** y **color** asociado.

```

1 // Declaracin de los tipos con: (nombre API, recurso imagen mipmap, recurso color)
2 public enum TipoPokemonEnum {
3     normal("normal", R.mipmap.normal, R.color.normalColor),
4     fuego("fire", R.mipmap.fuego, R.color.fuegoColor),
5     agua("water", R.mipmap.agua, R.color.aguaColor),
6     electrico("electric", R.mipmap.electrico, R.color.electricoColor),
7     planta("grass", R.mipmap.planta, R.color.plantaColor),
8     hielo("ice", R.mipmap.hielo, R.color.hieloColor),
9     lucha("fighting", R.mipmap.lucha, R.color.luchaColor),
10    veneno("poison", R.mipmap.veneno, R.color.venenoColor),
11    tierra("ground", R.mipmap.tierra, R.color.tierraColor),
12    volador("flying", R.mipmap.volador, R.color.voladorColor),
13    psiquico("psychic", R.mipmap.psiquico, R.color.psiquicoColor),
14    bicho("bug", R.mipmap.bicho, R.color.bichoColor),
15    roca("rock", R.mipmap.roca, R.color.rocaColor),
16    fantasma("ghost", R.mipmap.fantasma, R.color.fantasmaColor),
17    dragon("dragon", R.mipmap.dragon, R.color.dragonColor),
18    siniestro("dark", R.mipmap.siniestro, R.color.siniestroColor),
19    acero("steel", R.mipmap.acero, R.color.aceroColor),
20    hada("fairy", R.mipmap.hada, R.color.hadaColor);
21 }

```

Constructor Inicializa los atributos internos del enum: nombre, imagen y color.

```

1 // Atributos internos del enum
2 private final String nombre; // nombre del tipo segn la API (ej. "water")

```

```

3 private final int imagenResId; // id del recurso mipmap para el icono del tipo
4 private final int colorResId; // id del recurso color asociado al tipo
5
6 // Constructor del enum que asigna los valores a cada instancia
7 TipoPokemonEnum(String nombre, int imagenResId, int colorResId) {
8     this.nombre = nombre; // asigna nombre
9     this.imagenResId = imagenResId; // asigna recurso de imagen
10    this.colorResId = colorResId; // asigna recurso de color
11 }

```

fromName(String name) Busca el tipo cuyo nombre coincide con el **parámetro** y devuelve la instancia **correspondiente**, o null si no se encuentra.

```

1 // Busca una instancia del enum por su nombre (case-insensitive)
2 public static TipoPokemonEnum fromName(String name) {
3     for (TipoPokemonEnum tipo : values()) {
4         // compara el nombre interno con el parametro ignorando mayusculas
5         if (tipo.nombre.equalsIgnoreCase(name)) {
6             return tipo; // devuelve la coincidencia encontrada
7         }
8     }
9     return null; // devuelve null si no hay coincidencia
10 }

```

getNombre() Devuelve el nombre del tipo (en inglés).

```

1 // Getter del nombre (uso interno, p. ej. para comparar con la API)
2 public String getNombre() {
3     return nombre;
4 }

```

getImagenResId() Devuelve el ID del recurso de imagen asociado al tipo.

```

1 // Getter del recurso de imagen (mipmap) para mostrar el icono del tipo
2 public int getImagenResId() {
3     return imagenResId;
4 }

```

getColorResId() Devuelve el ID del color asociado al tipo.

```

1 // Getter del recurso de color (colors.xml) usado para teir fondos o acentos
2 public int getColorResId() {
3     return colorResId;
4 }

```


6.4. LandingActivity.java

Ubicación

`com.example.pokedex.View`

6.4.1. Descripción general

Actividad de lanzamiento que muestra una **barra de progreso** mientras la app descarga datos. **Ordena** la Pokédex por **número** y muestra un **botón** para entrar en la **vista principal** cuando la carga termina.

6.4.2. Funciones

onCreate(Bundle) Inicia la descarga mediante `API.obtainAllPokemon`, muestra/oculta progreso y botón, ordena la lista y configura el botón para acceder a la vista principal.

```
1 // Ciclo de vida onCreate: prepara UI, lanza la descarga y configura el botn de acceso
2 @Override
3 protected void onCreate(Bundle savedInstanceState) {
4     super.onCreate(savedInstanceState);
5     EdgeToEdge.enable(this);
6     setContentView(R.layout.activity_landing); // Carga el layout de la actividad
7
8     // Referencias a vistas del layout
9     btnAcceder = findViewById(R.id.btnAcceder);
10    progressBar = findViewById(R.id.progressBar);
11
12    // Mostrar estado de carga inicial: ocultar botn y mostrar progressbar
13    btnAcceder.setVisibility(View.INVISIBLE);
14    progressBar.setVisibility(View.VISIBLE);
15
16    // Inicia la descarga de datos desde la API; callback se ejecuta en el hilo UI
17    API.obtainAllPokemon(this, () -> runOnUiThread(() -> {
18        datosCargados = true; // Marca que los datos ya se han descargado
19
20        // Ordena la Pokdex por nmero antes de mostrarla
21        Collections.sort(API.getMyPokedex(), Comparator.comparingInt(Pokemon::getNumber
22        ));
23
24        // Oculta la barra de carga y muestra el botn para entrar
25        progressBar.setVisibility(View.GONE);
26        btnAcceder.setVisibility(View.VISIBLE);
27    }));
28
29    // Configura el comportamiento del botn: deshabilitar mltiples clics y esperar a
    // que los datos estn listos
    btnAcceder.setOnClickListener(v -> {
```

```

30     btnAcceder.setEnabled(false); // Previene mltiples pulsaciones
31     btnAcceder.setText("Accediendo..."); // Feedback visual
32     esperarCargaYEntrar(); // Espera hasta que datosCargados sea true y lanza la
        siguiente actividad
33     });
34 }

```

esperarCargaYEntrar() Bucle con Handler que espera hasta que datosCargados sea true y lanza GeneralPokedex.

```

1 // Espera activa (no bloqueante) hasta que los datos estn cargados y lanza la
    actividad principal
2 private void esperarCargaYEntrar() {
3     if (datosCargados) {
4         // Si ya estn cargados, inicia la actividad inmediatamente
5         startActivity(new Intent(this, GeneralPokedex.class));
6         finish(); // Cierra LandingActivity para no volver atrs
7     } else {
8         // Si no estn listos, crea un Handler que reintenta cada 500ms
9         Handler handler = new Handler();
10        handler.postDelayed(new Runnable() {
11            @Override
12            public void run() {
13                if (datosCargados) {
14                    // Cuando los datos estn listos, inicia la actividad y termina esta
15                    startActivity(new Intent(LandingActivity.this, GeneralPokedex.class)
16                        );
17                    finish();
18                } else {
19                    // Reprograma la comprobacin tras 500ms
20                    handler.postDelayed(this, 500);
21                }
22            }, 500); // Primer retraso inicial de 500ms
23        }
24    }

```

6.5. GeneralPokedex.java

Ubicación

com.example.pokedex.View

6.5.1. Descripción general

Actividad principal que muestra la lista de Pokémon en un RecyclerView con GridLayoutManager. Implementa **búsqueda en tiempo real** y **carga** paginada en **bloques** de 50 elementos.

6.5.2. Funciones

onCreate(Bundle) Obtiene la lista completa de `API.getMyPokedex()`, inicializa el adaptador y el `SearchView` para búsqueda en tiempo real.

```
1 // Ciclo de vida onCreate: prepara la UI, obtiene datos y configura bsqueda y
  adaptador
2 @Override
3 protected void onCreate(Bundle savedInstanceState) {
4     super.onCreate(savedInstanceState);
5     setContentView(R.layout.activity_general_pokedex); // Carga el layout principal
6
7     // Obtiene la lista completa de la API (ya poblada en LandingActivity)
8     pokedexCompleta = API.getMyPokedex();
9
10    // Lista que se mostrar en pantalla (filtrada/paginada)
11    pokedexFiltrada = new ArrayList<>();
12    loadNextBlock(); // Carga el primer bloque de elementos para mejorar el rendimiento
      inicial
13
14    // Configura RecyclerView con un GridLayout de 3 columnas
15    RecyclerView recyclerView = findViewById(R.id.pokedexRecycler);
16    recyclerView.setLayoutManager(new GridLayoutManager(this, 3));
17
18    // Crea y asigna el adaptador con la lista filtrada
19    adapter = new PokemonAdapter(pokedexFiltrada, this);
20    recyclerView.setAdapter(adapter);
21
22    // Configura el SearchView para filtrar por nombre o nmero en tiempo real
23    SearchView searchView = findViewById(R.id.SearchBar);
24    searchView.setOnQueryTextListener(new SearchView.OnQueryTextListener() {
25        @Override
26        public boolean onQueryTextSubmit(String query) {
27            return false; // No se usa submit; se filtra en onQueryTextChange
28        }
29
30        @Override
31        public boolean onQueryTextChange(String newText) {
32            pokedexFiltrada.clear(); // Limpia la lista mostrada
33
34            if (newText.isEmpty()) {
35                // Si el texto est vaco , reinicia la paginacin y carga el primer bloque
36                currentIndex = 0;
37                loadNextBlock();
38            } else {
39                // Filtra la lista completa comprobando nombre (case-insensitive) y
              nmero
40                for (Pokemon p : pokedexCompleta) {
41                    if (p.getName().toLowerCase().contains(newText.toLowerCase()) ||
```

```

42         String.valueOf(p.getNumber()).contains(newText)) {
43             pokedexFiltrada.add(p);
44         }
45     }
46 }
47
48 adapter.notifyDataSetChanged(); // Notifica cambios al adaptador para
    refrescar UI
49 return true;
50 }
51 });
52 }

```

loadNextBlock() Añade el siguiente bloque de 50 Pokémon a la lista mostrada para reducir el trabajo inicial de rendering.

```

1 // Carga el siguiente bloque de 50 elementos desde pokedexCompleta a pokedexFiltrada
2 private void loadNextBlock() {
3     int endIndex = Math.min(currentIndex + 50, pokedexCompleta.size()); // No exceder
    tamaño total
4     for (int i = currentIndex; i < endIndex; i++) {
5         pokedexFiltrada.add(pokedexCompleta.get(i)); // Aade cada Pokmon al bloque
    mostrado
6     }
7     currentIndex = endIndex; // Actualiza ndice para la siguiente carga
8 }

```

6.6. InfoPokemon.java

6.6.1. Ubicación

com.example.pokedex.View

6.6.2. Descripción general

Actividad de detalle que muestra la **ficha completa** de un Pokémon seleccionado: imagen, nombre, peso, tamaño, historia y hasta dos tipos. Aplica **color de fondo** e **íconos** según el tipo principal mediante TipoPokemonEnum.

6.6.3. Funciones

onCreate(Bundle) Recibe datos por Intent, llena las vistas, carga la **imagen** con **Glide** y aplica los **recursos visuales de tipo**. También configura el **botón de regreso** a la vista principal.

```

1 // Ciclo de vida onCreate: configura UI, recupera datos del Intent y aplica estilos
    por tipo

```

```

2  @Override
3  protected void onCreate(Bundle savedInstanceState) {
4      super.onCreate(savedInstanceState);
5      setContentView(R.layout.activity_info_pokemon); // Carga layout de detalle
6
7      // Recupera datos enviados desde la actividad previa
8      Intent intent = getIntent();
9      String nombre = intent.getStringExtra("nombre"); // Nombre del Pokmon
10     String peso = intent.getStringExtra("peso"); // Peso en kg (string)
11     String tamao = intent.getStringExtra("tamao"); // Altura en m (string)
12     String historia = intent.getStringExtra("historia"); // Texto descriptivo en
        espaol
13     String imageUrl = intent.getStringExtra("imagen"); // URL de la imagen oficial
14     String[] tipos = intent.getStringArrayExtra("tipos"); // Array de tipos (puede
        tener 1 o 2)
15
16     // Referencias a vistas del layout
17     TextView nombreView = findViewById(R.id.Nombre);
18     TextView pesoView = findViewById(R.id.peso);
19     TextView tamaoView = findViewById(R.id.tamao);
20     TextView historiaView = findViewById(R.id.historia);
21     ImageView imagenView = findViewById(R.id.Pokemon);
22     ImageView tipo1 = findViewById(R.id.tipo1);
23     ImageView tipo2 = findViewById(R.id.tipo2);
24     ConstraintLayout fondo = findViewById(R.id.infoPokemon);
25
26     // Asigna valores a las vistas de texto
27     nombreView.setText(nombre);
28     pesoView.setText("Peso: " + peso + " kg");
29     tamaoView.setText("Tamao: " + tamao + " m");
30     historiaView.setText(historia);
31
32     // Carga la imagen remota usando Glide (gestiona caching y threading)
33     Glide.with(this).load(imageUrl).into(imagenView);
34
35     // Si hay tipos, aplica iconos y color de fondo segn el tipo principal
36     if (tipos != null && tipos.length > 0) {
37         // Obtiene el enum que corresponde al nombre del tipo (por ejemplo "water")
38         TipoPokemonEnum tipoPrincipal = TipoPokemonEnum.fromName(tipos[0]);
39         if (tipoPrincipal != null) {
40             // Asigna icono del tipo principal y aplica color de fondo asociado
41             tipo1.setImageResource(tipoPrincipal.getImagenResId());
42             fondo.setBackgroundColor(getColor(tipoPrincipal.getColorResId()));
43         }
44
45         // Si existe un segundo tipo, mustralos; si no, oculta la vista correspondiente
46         if (tipos.length > 1) {
47             TipoPokemonEnum tipoSecundario = TipoPokemonEnum.fromName(tipos[1]);

```

```

48         if (tipoSecundario != null) {
49             tipo2.setImageResource(tipoSecundario.getImagenResId()); // Icono del
                tipo secundario
50         } else {
51             tipo2.setVisibility(View.GONE); // Oculta si no se reconoce el tipo
52         }
53     } else {
54         tipo2.setVisibility(View.GONE); // Oculta si no hay segundo tipo
55     }
56 }
57
58 // Configura el botn de volver a la vista principal (GeneralPokedex)
59 ImageButton backButton = findViewById(R.id.home);
60 backButton.setOnClickListener(v -> {
61     Intent volverIntent = new Intent(InfoPokemon.this, GeneralPokedex.class);
62     startActivity(volverIntent); // Lanza la actividad principal
63     finish(); // Cierra la actividad de detalle para no apilarla
64 });
65 }

```

6.7. PokemonAdapter.java

Ubicación

com.example.pokedex.View

6.7.1. Descripción general

Adaptador de RecyclerView que infla `item_pokemon.xml`, asigna **nombre**, **número** e **imagen** y maneja el clic para abrir `InfoPokemon` pasando los datos por `Intent`.

6.7.2. Funciones

Constructor Inicializa el adaptador con la lista de Pokémon y el contexto de la actividad.

```

1 // Constructor: guarda la lista de Pokmon y el contexto para inflar vistas y lanzar
    Intents
2 public PokemonAdapter(ArrayList<Pokemon> pokemons, Context context) {
3     this.pokemons = pokemons;
4     this.context = context;
5 }

```

onCreateViewHolder Infla el layout de cada tarjeta visual desde `item_pokemon.xml`.

```

1 // Infla el layout de la tarjeta y crea un ViewHolder para l
2 @Override

```

```

3 public PokemonViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {
4     View view = LayoutInflater.from(context).inflate(R.layout.item_pokemon, parent,
5         false);
6     return new PokemonViewHolder(view);
7 }

```

onBindViewHolder Asigna los datos del Pokémon a la **tarjeta**: número, nombre, imagen y configuración del clic para abrir InfoPokemon.

```

1 // Vincula datos del Pokmon a la tarjeta visual y configura el clic para abrir el
  // detalle
2 @Override
3 public void onBindViewHolder(PokemonViewHolder holder, int position) {
4     Pokemon p = pokemons.get(position); // Obtiene el Pokmon de la posicin
5
6     // Formatea y muestra el nmero como No.001
7     holder.pokemonNumber.setText("No." + String.format("%03d", p.getNumber()));
8     holder.pokemonName.setText(p.getName()); // Muestra el nombre
9
10    // Carga la imagen remota con Glide (thumbnail para mejorar percepcin de carga)
11    Glide.with(context)
12        .load(p.getImage())
13        .thumbnail(0.1f) // primero muestra miniatura
14        .dontAnimate() // evita animaciones que pueden chocar con RecyclerView
15        .into(holder.pokemonImage);
16
17    // Maneja el clic en la tarjeta: crea Intent con todos los datos y lanza
    // InfoPokemon
18    holder.itemView.setOnClickListener(view -> {
19        Intent intent = new Intent(context, InfoPokemon.class);
20        intent.putExtra("nombre", p.getName());
21        intent.putExtra("imagen", p.getImage());
22        intent.putExtra("peso", String.valueOf(p.getPeso()));
23        intent.putExtra("tamao", String.valueOf(p.getTamao()));
24        intent.putExtra("historia", p.getHistoria());
25        intent.putExtra("tipos", p.getTipos().toArray(new String[0]));
26        context.startActivity(intent);
27    });
28 }

```

getItemCount Retorna el tamaño de la lista de Pokémon.

```

1 // Devuelve cuntos elementos hay en la lista (usado por RecyclerView)
2 @Override
3 public int getItemCount() {
4     return pokemons.size();
5 }

```

PokemonViewHolder Clase interna que representa cada tarjeta visual y conecta los elementos del layout.

```
1 // ViewHolder que cachea referencias a vistas dentro de item_pokemon.xml para
   rendimiento
2 public static class PokemonViewHolder extends RecyclerView.ViewHolder {
3     ImageView pokemonImage; // Imagen principal del Pokmon
4     TextView pokemonNumber; // Texto con el nmero formateado
5     TextView pokemonName; // Texto con el nombre
6
7     public PokemonViewHolder(View itemView) {
8         super(itemView);
9         // Conecta referencias a vistas para evitar findViewById repetidos en bind
10        pokemonImage = itemView.findViewById(R.id.pokemonImage);
11        pokemonNumber = itemView.findViewById(R.id.pokemonNumber);
12        pokemonName = itemView.findViewById(R.id.pokemonName);
13    }
14 }
```

6.8. AndroidManifest.xml

Ubicación

/app/src/main/AndroidManifest.xml

6.8.1. Descripción general

Define **permisos** (**Internet**), actividades y metadatos de la aplicación (**íconos**, tema, reglas de backup). Declara la **actividad** de **lanzamiento** (**LandingActivity**) con intent-filter **MAIN+LAUNCHER**.

6.8.2. Código comentado

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:tools="http://schemas.android.com/tools">
4
5     <!-- Permiso para acceder a Internet -->
6     <uses-permission android:name="android.permission.INTERNET" />
7
8     <application
9         android:allowBackup="true"
10        android:dataExtractionRules="@xml/data_extraction_rules"
11        android:fullBackupContent="@xml/backup_rules"
12        android:icon="@mipmap/ic_launcher"
13        android:label="@string/app_name"
14        android:roundIcon="@mipmap/ic_launcher_round"
```



```

15     android:supportsRtl="true"
16     android:theme="@style/Theme.Pokedex">
17
18     <!-- Actividad que muestra la informacin detallada de un Pokmon -->
19     <activity
20         android:name=".View.InfoPokemon"
21         android:exported="false" />
22
23     <!-- Actividad principal con la lista de Pokmon -->
24     <activity
25         android:name=".View.GeneralPokedex"
26         android:exported="false" />
27
28     <!-- Actividad de inicio que carga los datos -->
29     <activity
30         android:name=".View.LandingActivity"
31         android:exported="true" >
32         <intent-filter>
33             <!-- Define que esta actividad se lanza al abrir la app -->
34             <action android:name="android.intent.action.MAIN" />
35             <category android:name="android.intent.category.LAUNCHER" />
36         </intent-filter>
37     </activity>
38 </application>
39
40 </manifest>

```

6.9. Layouts XML

6.9.1. activity_general_pokedex.xml

Ubicación res/layout/activity_general_pokedex.xml

Descripción general Layout de la vista principal con `SearchView` y `RecyclerView`. Fondo decorativo.

Fragmento comentado

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <!-- Layout principal con fondo decorativo -->
3 <androidx.constraintlayout.widget.ConstraintLayout
4     xmlns:android="http://schemas.android.com/apk/res/android"
5     xmlns:app="http://schemas.android.com/apk/res-auto"
6     xmlns:tools="http://schemas.android.com/tools"
7     android:id="@+id/main"
8     android:layout_width="match_parent"
9     android:layout_height="match_parent"
10    android:background="@mipmap/background"

```

```

11     tools:context=".View.GeneralPokedex">
12
13     <!-- Barra de bsqueda para filtrar Pokmon -->
14     <SearchView
15         android:id="@+id/SearchBar"
16         android:layout_width="match_parent"
17         android:layout_height="55dp"
18         android:layout_margin="16dp"
19         android:background="@mipmap/barra_buscar"
20         android:iconifiedByDefault="false"
21         android:queryHint="Busca tu Pokmon"
22         app:layout_constraintTop_toTopOf="parent"
23         app:layout_constraintStart_toStartOf="parent"
24         app:layout_constraintEnd_toEndOf="parent" />
25
26     <!-- Lista de tarjetas de Pokmon -->
27     <androidx.recyclerview.widget.RecyclerView
28         android:id="@+id/pokedexRecycler"
29         android:layout_width="0dp"
30         android:layout_height="0dp"
31         android:padding="8dp"
32         app:layout_constraintTop_toBottomOf="@id/SearchBar"
33         app:layout_constraintBottom_toBottomOf="parent"
34         app:layout_constraintStart_toStartOf="parent"
35         app:layout_constraintEnd_toEndOf="parent" />
36 </androidx.constraintlayout.widget.ConstraintLayout>

```

6.9.2. activity_info_pokemon.xml

Ubicación res/layout/activity_info_pokemon.xml

Descripción general Ficha detalle con imagen, nombre, especificaciones y tipos; fondo dinámico según tipo.

Fragmento comentado

```

1 <!-- Layout principal con fondo dinámico -->
2 <androidx.constraintlayout.widget.ConstraintLayout
3     android:id="@+id/infoPokemon"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent"
6     android:background="#B9EDDF">
7
8     <!-- Fondo blanco decorativo detrás del contenido -->
9     <ImageView
10         android:id="@+id/imageView5"
11         android:layout_width="412dp"
12         android:layout_height="678dp"

```

```

13         android:background="@mipmap/rectangulo"
14         android:contentDescription="@string/fondoblanco"
15     ... />
16
17 <!-- Imagen del Pokmon -->
18 <ImageView
19     android:id="@+id/Pokemon"
20     android:layout_width="259dp"
21     android:layout_height="319dp"
22     android:contentDescription="@string/imagenPokemon"
23     app:srcCompat="@mipmap/_07__1_" />
24
25 <!-- Nombre del Pokmon -->
26 <TextView
27     android:id="@+id/Nombre"
28     android:text="@string/nombre"
29     android:textSize="20sp"
30     android:textStyle="bold"
31     android:textColor="@color/black" />
32
33 <!-- Peso del Pokmon -->
34 <TextView
35     android:id="@+id/peso"
36     android:text="@string/peso"
37     android:textSize="20sp"
38     android:textStyle="bold"
39     android:textColor="@color/black" />
40
41 <!-- Tamao del Pokmon -->
42 <TextView
43     android:id="@+id/tamao"
44     android:text="@string/tama_o"
45     android:textSize="20sp"
46     android:textStyle="bold"
47     android:textColor="@color/black" />
48
49 <!-- Historia del Pokmon -->
50 <TextView
51     android:id="@+id/historia"
52     android:text="@string/textview"
53     android:textSize="18sp"
54     android:textColor="@color/black" />
55
56 <!-- Iconos de tipo -->
57 <ImageView android:id="@+id/tipo1" app:srcCompat="@android:drawable/
    ic_notification_overlay" />
58 <ImageView android:id="@+id/tipo2" app:srcCompat="@android:drawable/
    ic_notification_overlay" />

```

```

59
60 <!-- Botn para volver a la pantalla principal -->
61 <ImageButton
62     android:id="@+id/home"
63     android:srcCompat="@mipmap/home2"
64     android:background="@android:color/transparent"
65     android:contentDescription="@string/back" />
66 </androidx.constraintlayout.widget.ConstraintLayout>

```

6.9.3. activity_landing.xml

Ubicación res/layout/activity_landing.xml

Descripción general Pantalla inicial con **botón** de acceso y **progressbar** visible durante la carga.

Fragmento comentado

```

1 <!-- Layout principal con fondo decorativo -->
2 <androidx.constraintlayout.widget.ConstraintLayout
3     android:id="@+id/main"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent"
6     android:background="@mipmap/fondoandroid"
7     android:foregroundTint="#509276"
8     tools:context=".View.LandingActivity">
9
10 <!-- Botn para acceder a la Pokdex -->
11 <Button
12     android:id="@+id/btnAcceder"
13     android:layout_width="273dp"
14     android:layout_height="65dp"
15     android:backgroundTint="#b9e0cf"
16     android:drawableLeft="@mipmap/iconopokedex4"
17     android:text="@string/accede_a_tu_pokedex"
18     android:textColor="#337157"
19     android:visibility="invisible"
20     app:cornerRadius="30dp"
21     app:iconSize="10dp"
22     app:layout_constraintBottom_toBottomOf="parent"
23     app:layout_constraintEnd_toEndOf="parent"
24     app:layout_constraintStart_toStartOf="parent"
25     app:layout_constraintTop_toTopOf="parent"
26     app:layout_constraintVertical_bias="0.634" />
27
28 <!-- Barra de progreso que se muestra mientras se descargan los datos -->
29 <ProgressBar
30     android:id="@+id/progressBar"

```

```

31     android:layout_width="60dp"
32     android:layout_height="60dp"
33     android:layout_marginTop="24dp"
34     android:indeterminateTint="#2B2D30"
35     android:visibility="gone"
36     app:layout_constraintEnd_toEndOf="parent"
37     app:layout_constraintStart_toStartOf="parent"
38     app:layout_constraintTop_toBottomOf="@id/btnAcceder" />
39 </androidx.constraintlayout.widget.ConstraintLayout>

```

6.9.4. item_pokemon.xml

Ubicación res/layout/item_pokemon.xml

Descripción general Tarjeta individual usada por el adaptador: imagen, número y nombre.

Fragmento comentado

```

1  <!-- Tarjeta individual de Pokmon -->
2  <androidx.constraintlayout.widget.ConstraintLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      xmlns:app="http://schemas.android.com/apk/res-auto"
5      android:layout_width="wrap_content"
6      android:layout_height="wrap_content"
7      android:layout_margin="8dp"
8      android:background="@drawable/pokemon_card_background"
9      android:padding="8dp">
10
11     <!-- Imagen del Pokmon -->
12     <ImageView
13         android:id="@+id/pokemonImage"
14         android:layout_width="100dp"
15         android:layout_height="100dp"
16         android:layout_marginBottom="8dp"
17         android:background="@drawable/pokemon_image_background"
18         android:scaleType="centerInside"
19         app:layout_constraintTop_toTopOf="parent"
20         app:layout_constraintStart_toStartOf="parent"
21         app:layout_constraintEnd_toEndOf="parent" />
22
23     <!-- Nmero del Pokmon -->
24     <TextView
25         android:id="@+id/pokemonNumber"
26         android:layout_width="wrap_content"
27         android:layout_height="wrap_content"
28         android:text="No.001"
29         android:textColor="@color/black"

```

```

30     android:textSize="14sp"
31     android:textStyle="bold"
32     app:layout_constraintTop_toBottomOf="@id/pokemonImage"
33     app:layout_constraintStart_toStartOf="

```

6.10. colors.xml

Ubicación res/values/colors.xml

Descripción Define colores base y **colores** específicos por **tipo** de Pokémon que se usan en la interfaz y en TipoPokemonEnum.

Contenido

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <resources>
3      <!-- Colores base del tema -->
4      <color name="purple_200">#FFBB86FC</color> <!-- Morado claro -->
5      <color name="purple_500">#FF6200EE</color> <!-- Morado medio -->
6      <color name="purple_700">#FF3700B3</color> <!-- Morado oscuro -->
7      <color name="teal_200">#FF03DAC5</color> <!-- Verde azulado claro -->
8      <color name="teal_700">#FF018786</color> <!-- Verde azulado oscuro -->
9      <color name="black">#FF000000</color> <!-- Negro -->
10     <color name="white">#FFFFFFFF</color> <!-- Blanco -->
11
12     <!-- Colores personalizados por tipo de Pokmon -->
13     <color name="normalColor">#d6d5c6</color> <!-- Tipo normal -->
14     <color name="fuegoColor">#f6c097</color> <!-- Tipo fuego -->
15     <color name="aguaColor">#B8BFFF</color> <!-- Tipo agua -->
16     <color name="electricoColor">#FFEA92</color> <!-- Tipo elctrico -->
17     <color name="plantaColor">#9DE57F</color> <!-- Tipo planta -->
18     <color name="hieloColor">#C3EDF5</color> <!-- Tipo hielo -->
19     <color name="luchaColor">#E37F7F</color> <!-- Tipo lucha -->
20     <color name="venenoColor">#D67FE5</color> <!-- Tipo veneno -->
21     <color name="tierraColor">#F2DDB0</color> <!-- Tipo tierra -->
22     <color name="voladorColor">#D3C7FE</color> <!-- Tipo volador -->
23     <color name="psiquicoColor">#FEA7CD</color> <!-- Tipo psiquico -->
24     <color name="bichoColor">#CDE18F</color> <!-- Tipo bicho -->
25     <color name="rocaColor">#E1C87F</color> <!-- Tipo roca -->
26     <color name="fantasmaColor">#B29CC9</color> <!-- Tipo fantasma -->
27     <color name="dragonColor">#B290F6</color> <!-- Tipo dragn -->
28     <color name="siniestroColor">#A3A3A3</color> <!-- Tipo siniestro -->
29     <color name="aceroColor">#D2D3E2</color> <!-- Tipo acero -->
30     <color name="hadaColor">#FCD2E8</color> <!-- Tipo hada -->
31 </resources>

```